Unit 5 Results Document

Ling Qingyang s2471545

Task 4.1 Likelihood Grid

Initially, wider parameter ranges resulted in sharp spikes in the likelihood grids. To properly visualize the interesting structure of the likelihood distribution, the ranges were narrowed down—particularly for H_0, which exhibits a highly constrained, narrow likelihood peak.
The final chosen ranges and grid settings are:

- Ω_m **range**: $[0.20, 0.55]$
- Ω_Λ **range**: $[0.50, 1.15]$
- H_0 **range**: $[71.5, 73.5]$ km s^{-1} Mpc^{-1}
- **Samples per axis**: 30
- **Integration steps**: 500

The total computation evaluated $30 \times 30 \times 30 = 27,000$ grid points. The likelihood calculations took **20.77 seconds** to complete.
The maximum likelihood (best-fit) point found on this grid is:

- $\Omega_m = 0.35690$
- $\Omega_\Lambda = 0.83621$
- $H_0 = 72.12069$
- $\log L = -516.37933$

The corresponding 2D and 1D marginalized likelihood plots generated from this grid are presented in Figure 1 and Figure 2 below:

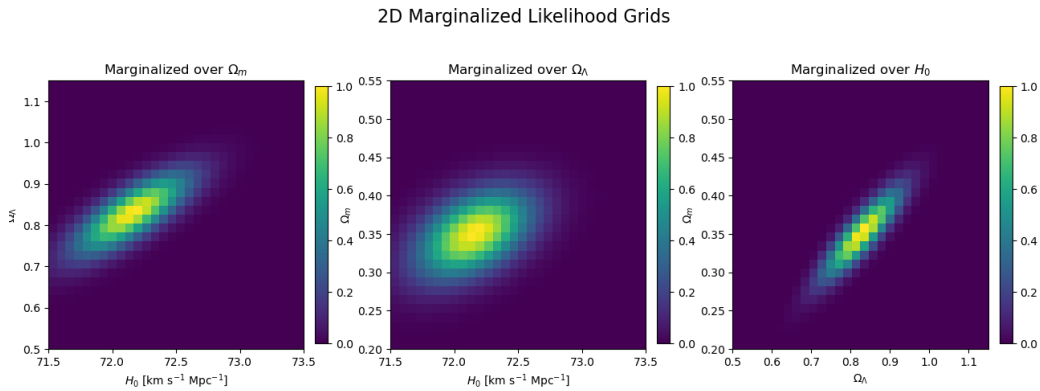

Figure 1: 2D Marginalized Likelihood Grids for the cosmological parameters.

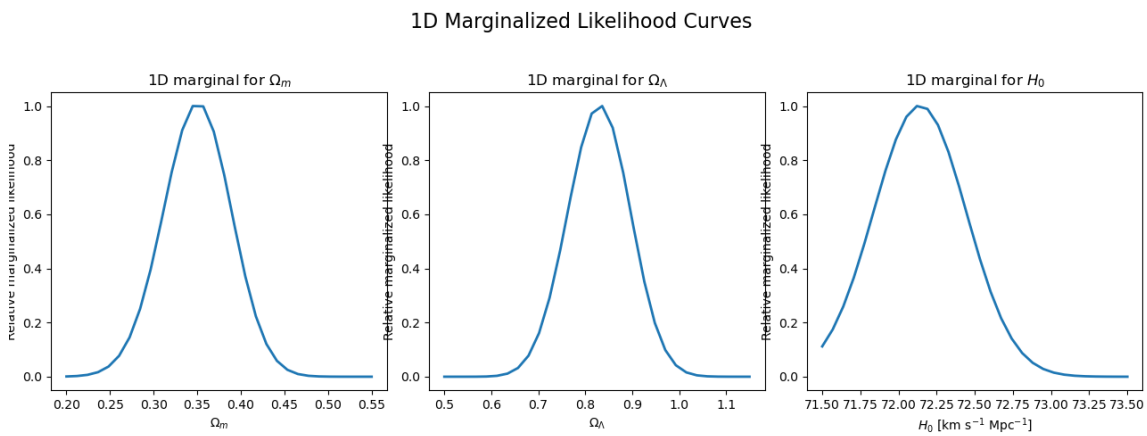

Figure 2: 1D Marginalized Likelihood Curves showing the probability distributions.

The best-fit region lies comfortably within the chosen parameter ranges rather than at the edges.

Task 4.2 Metropolis Algorithm Plan

1. Class Initialization (`__init__`)

The class will be initialized with the following inputs:

- `log_likelihood_function`: A callable function that takes a parameter array θ and returns the corresponding log-likelihood value.
- `initial_theta`: A NumPy array representing the starting point in parameter space.

- `step_sizes`: A NumPy array containing the standard deviations of the Gaussian proposal distribution for each parameter.
- `random_seed` (optional): An integer seed for reproducibility.

During initialization, the class will store the current parameter vector, evaluate the log-likelihood at the starting point, and determine the dimensionality of the problem from the length of `initial_theta`.

2. Class Attributes (Internal State)

The class will maintain the following internal attributes:

- `self.current_theta`: The current parameter vector in the Markov chain.
- `self.current_logL`: The log-likelihood value at `self.current_theta`.
- `self.dimension`: The number of parameters in the problem.
- `self.step_sizes`: The Gaussian proposal widths for each parameter.
- `self.chain`: An array or list storing the full Markov chain, with shape $(n_{\text{steps}}, n_{\text{parameters}})$. This will include repeated states when a proposal is rejected, since the chain remains at the old point in that case.
- `self.logL_history`: A record of the log-likelihood value at every step. This is useful for identifying the burn-in period and checking whether the chain has settled into the high-probability region.
- `self.accepted_steps`: A counter for the number of accepted proposals.
- `self.total_steps`: A counter for the total number of Metropolis steps attempted.

3. Core Methods

(a) propose_step(self)

This method generates a candidate point θ' by adding a Gaussian random step to the current position. The proposal is defined as:

$$\theta' = \theta + \sigma \cdot \mathcal{N}(0, 1)$$

where each parameter is perturbed independently according to its corresponding step size.

(b) take_step(self)

This method performs one full iteration of the Metropolis algorithm:

1. Call `propose_step()` to generate a proposed parameter point θ'.
2. Evaluate the log-likelihood $\log L'$ at the proposed point.
3. Apply the Metropolis acceptance rule:

- If $\log L' > \log L_i$, accept the new point directly.
- If $\log L' < \log L_i$, generate a uniform random number $u \in (0, 1)$, and accept the proposal if:

$$\log u < \log L' - \log L_i$$

  This log-space comparison avoids numerical underflow.

4. If the proposal is accepted, update `self.current_theta` and `self.current_logL`. If the proposal is rejected, retain the old values.
5. Append the resulting state, whether new or repeated, to `self.chain`, and append the corresponding log-likelihood to `self.logL_history`.
6. Update the counters `self.accepted_steps` and `self.total_steps`.

4.2 Metropolis Analysis

The proposal step sizes were set to $\sigma_{\Omega_m} = 0.07$, $\sigma_{\Omega_\Lambda} = 0.1$, and $\sigma_{H_0} = 0.3$ as recommended.
The chain was run for a total of **50,000 steps**. The first **500 steps** were discarded as the burn-in period, which was confirmed visually by the log-likelihood trace plot (Figure 3) where the chain rapidly climbed from the initial guess and stabilized around the maximum likelihood region.

MCMC Diagnostics and Results

- **Total Computation Time**: 39.78 seconds
- **Acceptance Rate**: 8.88% (a bit low, but the chain still converged to the same high-likelihood region identified by the likelihood grid)
- **Best-fit point (Maximum log-likelihood)**: $\Omega_m = 0.35067$, $\Omega_\Lambda = 0.83098$, $H_0 = 72.16247$ ($\log L = -516.34791$)
- **Posterior Means (with 1σ std)**:
  - $\Omega_m = 0.35143 \pm 0.03852$
  - $\Omega_\Lambda = 0.83253 \pm 0.06556$
  - $H_0 = 72.15944 \pm 0.29615$

Comparison with Likelihood Grid

The Metropolis analysis successfully reproduced the parameter distributions identified by the likelihood grid, but with several distinct advantages. While the 3D grid method required manually determining narrow boundaries to avoid capturing "dead zones," the MCMC chain naturally concentrated its samples in the high-probability regions without rigid bounds.
Furthermore, because MCMC evaluates points in a continuous space rather than on discrete intersections, it discovered a marginally higher maximum log-likelihood (-516.347) compared to the grid's best point (-516.379). Visually, the 1D marginalized MCMC histograms (Figure 4) exhibit much smoother, higher-resolution Gaussian profiles than the 1D grid curves, confirming that the chain has properly mapped the likelihood's structure.

MCMC Plots

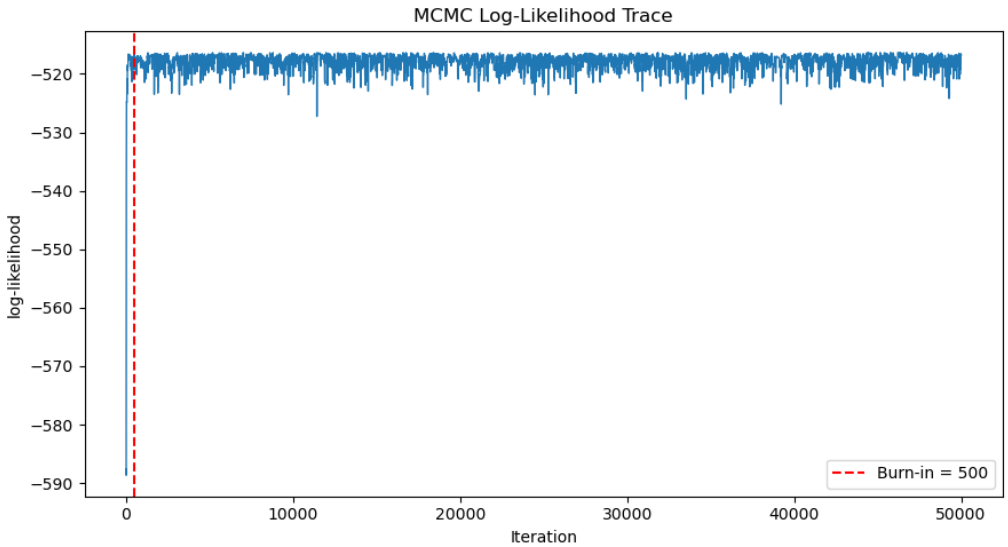

Figure 3: Log-Likelihood trace for the MCMC chain. The red dashed line indicates the 500-step burn-in cut.

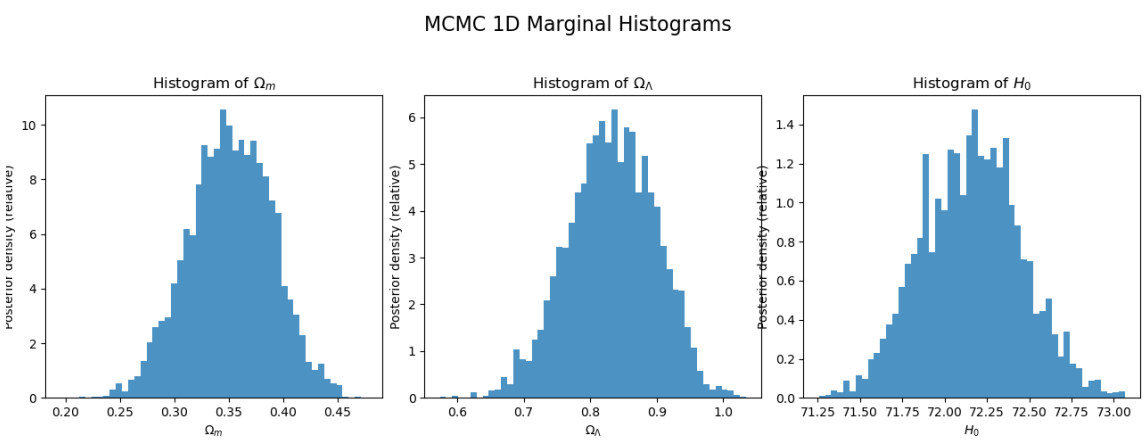

Figure 4: 1D Marginalized Posterior Histograms for the parameters after discarding the burn-in phase.

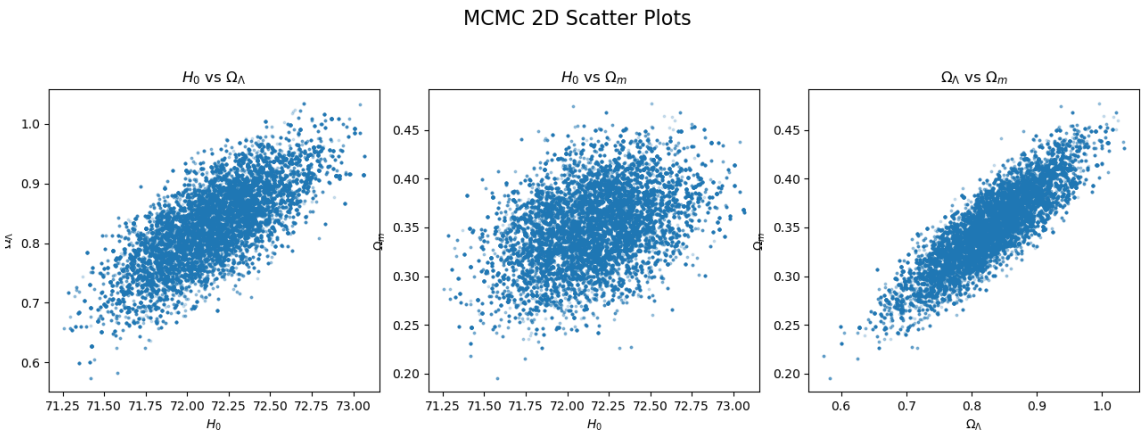

Figure 5: 2D Scatter Plots showing parameter correlations from the MCMC samples.